

Versatile Sketch-Based Attribute Filtering for Hybrid Vector Search

Adeel Aslam , Luca Gagliardelli , El Kindi Rezig , *Member, IEEE*, George Konstantinidis ,
and Giovanni Simonini

Abstract—This work addresses the problem of hybrid search in vector databases, which store vectors together with some property attributes. Given a query that consists of a vector and some restrictions on its property attributes, we want to retrieve approximate nearest neighbor vectors for the query while ensuring compliance with predicate conditions, such as point or range filters on a specific vector property attribute. The challenge is compounded by the need to balance two competing requirements: on one hand, ensuring high accuracy in the vector search by leveraging a similarity-based index that is independent of specific attributes, allowing it to serve all queries; on the other hand, the impracticality of replicating such a structure for each attribute or predicate condition. To address these challenges, we propose an agnostic, attribute popularity-aware solution for predicate filtering in approximate nearest neighbor (ANN) search, leveraging the efficiency of graph-based indexing structures for vectors. Our method begins by clustering nodes within the underlying graph structure and constructing lightweight in-memory sketches for the predicates. During query processing, the search selectively applies a two-hop traversal strategy only when necessary, guided by the attribute popularity within the identified cluster. Experimental evaluation across five benchmark datasets demonstrates that our approach consistently outperforms state-of-the-art methods.

Index Terms—Data sketches, hybrid query, nearest neighbor, predicate filtering, proximity graph, distance computation.

I. INTRODUCTION

VECTOR databases have gained prominence as a key technology for applications that involve unstructured data such as images, text, audio, and videos [1], [2], [3]. These databases store the data in the form of vector embeddings—dense numerical representations that encode the semantic content of unstructured items. Unlike traditional databases,

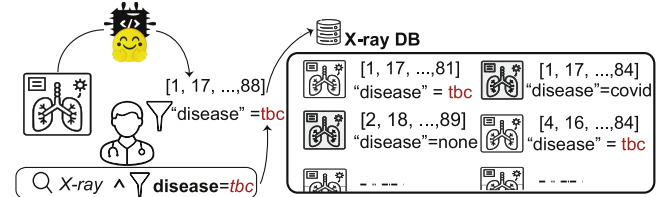


Fig. 1. A vector database (X-ray DB) stores.

vector databases facilitate *approximate nearest-neighbor* (ANN) search based on semantic similarity rather than exact matching [4], [5]. Additionally, their capability to manage vectors with thousands of dimensions, along with real-time query support, establishes them as a crucial component of contemporary machine learning-based workflows, including recommendation systems, where these systems can help with minimizing hallucinations from LLM [6], [7].

In real-world applications, many queries go beyond a simple similarity search on vectors. Alongside ANN retrieval, they may also include predicate conditions on structured data attributes. These are commonly referred to as hybrid queries or hybrid vector search. To illustrate this, consider a medical use case (Fig. 1). In this scenario, a doctor wants to retrieve similar X-ray scans from a patient's X-ray record and apply a filter based on the *disease* attribute. The doctor provides the X-ray scan to the X-ray DB system. The system then converts the scan into a vector embedding and searches the X-ray DB for similar records, applying a filter on the *disease* attribute.

A naïve approach to handling hybrid queries is to first filter data using the query's attribute predicates and then perform a brute-force search (pre-filtering) [14]. While this method ensures high accuracy, it becomes increasingly inefficient for low-selectivity queries as the dataset grows, leading to significant performance bottlenecks. To address this issue, several post-filtering methodologies have been proposed to enhance existing ANN techniques for hybrid queries, including product quantization (PQ) [4], [19], locality-sensitive hashing (LSH) [20], inverted file index (IVF) [21], and proximity graph (PG)-based approaches [7], [9]. Among these, PG-based methods provide a more favorable balance between execution efficiency and recall [13], [14]. Table I summarizes the properties of state-of-the-art PG-based techniques for hybrid query processing. Table II explains the notation used in Table I and throughout the paper.

Received 2 September 2025; revised 16 April 2026; accepted 27 June 2026. Date of publication 1 July 2026; date of current version 8 August 2026. This work was supported in part by Project "Data Quality for Responsible Data Science: Human in the Loop for Quality Data" under Grant CUP: 202248FWFS, in part by CINECA under Grant ISCRA B id-HP10BTGNQ_C, in part by Horizon Europe projects UPGAST under Grant 101093216, in part by DATAPACT under Grant 101189771, and in part by RAISE SUITE under Grant 101188337. Recommended for acceptance by S. Shang. (Corresponding author: Adeel Aslam.)

Adeel Aslam and George Konstantinidis are with the University of Southampton, SO17 1BJ Southampton, U.K. (e-mail: A.Aslam@soton.ac.uk; G.Konstantinidis@soton.ac.uk).

Luca Gagliardelli is with the eCampus University, 22060 Novedrate, Italy (e-mail: luca.gagliardelli@unicampus.it).

El Kindi Rezig is with the University of Utah, Salt Lake City UT 84112 USA (e-mail: elkindi.rezig@utah.edu).

Giovanni Simonini is with the University of Modena and Reggio Emilia, 41121 Modena, Italy (e-mail: simonini@unimore.it).

Digital Object Identifier 10.1109/TKDE.2026.3709125

TABLE I
COMPARISON OF PROXIMITY GRAPH-BASED HYBRID VECTOR SEARCH TECHNIQUES

	Graph Construction		Hybrid Query Type			Properties			
Method	Meta data.	Agn.	Pred. Agn.	Eq.	Range	Hybrid.	Construction Complexity	Space Complexity	Overhead
Ours	✓		✓	✓	✓	✓	$O(c(h \cdot n_s) + n \log n)$	$O(n \cdot M)$	In-memory sketches
iRange graph [8]	×		×	×	✓	×	$O(n \log n + s(n_s) \cdot \beta \log n_s)$	$O(n \cdot (M - \beta) \cdot \log n)$	Sorting and construction
Stitched Vamana [9]	✓		×	×	✓	✓	$O(n + L \cdot n_L \log n_L + n \cdot R)$	$O(n \cdot (M_{\text{stitched}} - \beta))$	Merging and construction
Filtered Vamana [9]	✓		×	×	✓	✓	$O(n \cdot (e_l) + n \cdot \beta \log n)$	$O(n \cdot (M + E))$	Starting points identification
SeRF [10]	×		×	×	✓	×	$O(n \log n + n \cdot \beta_r \log n)$	$O(n \cdot (M_{v,b,e} - \beta))$	Sorting and construction
UNIFY [11]	×		×	×	×	×	$O(n \log n + c \cdot n_s G[i] \log n_s)$	$O(c \cdot n_s \cdot G[i](M))$	Adjacency list memory
NPG [12]	✓		✓	✓	✓	✓	$O(n + n \cdot \beta_r \log n)$	$O(N \cdot (M - \beta))$	Distance computation
NaviX [13]	✓		✓	✓	✓	✓	$O(n \log n)$	$O(n \cdot M)$	Local selectivity
ACORN-1 [14]	✓		✓	✓	✓	✓	$O(n \log n)$	$O(n \cdot M)$	2-hop search
ACORN- γ [14]	✓		✓	✓	✓	✓	$O(n \cdot \beta_r \log n)$	$O(n \cdot (M_\gamma - \beta))$	Extended HNSW
TANN [15]	×		✓	✓	✓	×	$O(n \log^2 n)$	$O(n \cdot M^2)$	Neighbor tree edges
WST-Tree [16]	×		×	×	×	×	$O(n \log n + n \log^2 n)$	$O(n \cdot M \log n)$	Multi-level graphs
UNG [17]	✓		×	×	×	×	$O(L \log L + L(n_s \log n_s))$	$O(n \cdot L(M))$	Heavy pre-processing
TAG [18]	✓		×	✓	×	×	$O(L_f + n \cdot \beta_r \log n)$	$O(n \cdot (L + e_l) \cdot M)$	Label-node density

TABLE II
NOTATIONS

Notation	Description
n	Number of nodes (d-dimensional vectors)
M	Edges per node
h	Hash functions
c, s, n_s	Clusters, segments, and subset size
β, β_r	Pruning and extra edges cost
E	Entry points for labels
n_L	Nodes with metadata l
M_{stitched}, R	Total stitched edges and cost
$M_{v,b,e}$	Neighbors for range boundaries
N	Nodes with feature and attribute vectors
e_l	Entry points per label
$G[i]$	Adjacency list of node i
L, L_f	Labels and their frequencies
M_γ	Edges with expansion factor γ

In Table I, we point out that many PG-based, or simply, *graph-based* approaches, are specifically tailored for range or point filtering [8], [17]. These specialized solutions rely heavily on metadata when building the proximity graph, which can impact the retrieval of the closest object for unstructured parts of a query. Additionally, some methods are not metadata-agnostic and need a separate graph structure for each data attribute [10], [12]. In certain cases, the original graph is modified based on data attributes or predicate conditions, leading to increased memory overhead [9], [14]. Given these considerations, there is a need for a technique that supports predicate filtering while remaining independent of metadata. Ideally, such a method would use a simple proximity graph structure to minimize additional memory costs and enable a more efficient hybrid search.

Our contribution: In this paper, we propose a novel solution for hybrid queries that leverages a standard proximity graph, such as Hierarchical Navigable Small World (HNSW), due to its practicality and widespread adoption in both commercial and open-source systems, highly optimized and refined over time [2], [3], [22], [23]. Our objective is to provide an efficient solution for a wide range of hybrid queries by utilizing the proximity graph without incorporating metadata (attribute properties) or predicate conditions into the graph index. To achieve that, our solution employs lightweight in-memory data structures to capture the popularity of attributes, which remain independent of the proximity graph and are leveraged during query processing. This design ensures seamless compatibility with any HNSW implementations.

The main intuition behind our solution is to partition the HNSW graph into a series of logical clusters and to create space-efficient in-memory sketches of node attributes. These sketches enable a constant-time estimation of the significance of a query attribute within each cluster. Additionally, this estimation is utilized for graph traversal. Furthermore, we leverage the power of two-hop neighbors expansion of the identified nearest neighbors containing high-frequency attributes within the graph to enhance accuracy.

Overall contributions can be summarized as follows:

- We analyze the limitations and overheads of existing graph-based hybrid search methods (Sections I, II, and VI).
- We propose a clustering methodology based on HNSW layer-0, where each cluster is equipped with an in-memory predictor to estimate local attribute selectivity for guiding graph traversal (Section III).
- We develop a unified prediction framework that leverages both individual predictors or their combinations across clusters to efficiently estimate attribute popularity for query processing (Section IV).
- We introduce an adaptive traversal mechanism, guided by selectivity and graph exploration, to enable efficient higher-order search for nearest neighbors under query predicates (Section IV).
- We design a strategy for high-selectivity queries that uses cluster-level predictors to initiate search from targeted entry points (Section IV-D).
- We develop a lightweight two-level hash-based filtering mechanism, constructed during clustering, to reduce unnecessary disk I/O (Section IV-E).
- We evaluate our approach on multiple real-world datasets and demonstrate its effectiveness compared to state-of-the-art methods (Section V-C).

II. PRELIMINARIES

In this section, we present the fundamentals of hybrid queries and describe the in-memory data structures.

A. ANN Hybrid Query

Let us consider a dataset $S = \{s_1, s_2, s_3, \dots, s_n\}$, where each item in the dataset $s_i = \{v_i, L_i\}$ consists of a vector

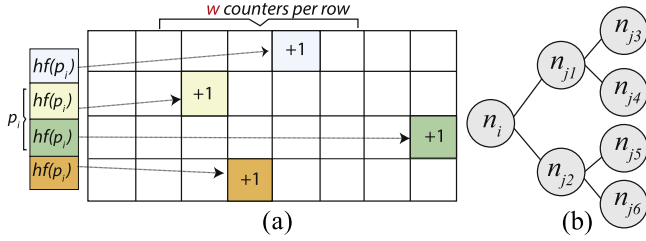


Fig. 2. (a) Count-Min sketch and (b) Two-hop neighbors.

v_i of dimension D , and a set of structured attributes $a_i = \{l_1, l_2, l_3, \dots, l_m\}$. For ANN, given a query q_i and an integer number K , the goal is to retrieve the top- K nearest vectors from the dataset S based on the distance function $\text{dist}(q_i, v_i)$ [5]. However, the *hybrid query* extends ANN by employing predicate filtering: the query is defined as $q_i = \{v_j, l_j\}$ and aims to identify the top- K nearest vectors from the dataset S that also satisfy the predicate condition between the query l_j and the dataset attribute l_i [1], [12].

Hierarchical navigable small world graph: For a new vector v_i , the insertion process begins by randomly selecting a layer 1 from the HNSW hierarchy. The algorithm then selects an entry point node n_e —either randomly or via a greedy heuristic—on layer 1 and performs a search to find the nearest neighbors to v_i from n_e . A corresponding node n_i for v_i is then inserted into the graph at this layer. To maintain efficient connectivity, several pruning strategies are applied for edge selection of node n_i . The total number of connections established for node n_i is controlled by the parameter M , while the breadth of the neighbor search during insertion is governed by the parameter efc . Moreover, the node n_i is inserted into all layers from layer 1 down to layer 0 using a similar procedure. The nearest neighbors found in layer 1 serve as entry points for the search in layer 1-1. Additionally, the search follows a similar procedure, and the search behavior is controlled by another parameter efs .

Two-hop neighbors: Let us consider a scenario where a node n_i is any node at layer 0 of the HNSW graph. We define the *two-hop neighbors* of n_i as the set of nodes that are nearest to n_i , including both nodes directly connected to n_i and those connected to its immediate neighbours, as depicted by Fig. 2(b). In proximity graphs, these two-hop neighbors are particularly important, as they help to identify the nearest neighbors [13], [14]. In this study, we explore the two-hop neighbors of each n_i for cluster construction (Section III-B) and predicate filtering (Section IV).

B. In-Memory Predictors

Count-Min sketch (CMS): We use CMS to estimate the popularity of structured query attributes, particularly for point filtering. It is a two-dimensional array consisting of d rows and w columns, where each row corresponds to a distinct hash function. When a new item is processed, it is hashed by each function, and the corresponding cells—determined by $h_i(p) \bmod w$ for the i -th hash function—are incremented by 1, as illustrated in Fig. 2(a). The minimum value across these cells is then taken as

Algorithm 1: Predictor-Based Hybrid Search.

Input: HNSW parameters (M, efc, efs), cluster size (C_s), dataset with attributes (S), query (q), and popularity threshold (Δ)

Output: Top- K nearest neighbors for q from S

// Graph construction and clustering (Steps 1-2) for all queries (offline).

- 1 Construct the HNSW graph using (M, efc) parameters;
- 2 Cluster nodes at layer 0 of HNSW and build in-memory sketches;
- 3 Query q on the HNSW graph;
- // Low-selectivity queries.
- 4 Identify the top- K nearest neighbors;
- 5 For each top- K item, evaluate the attribute popularity of q on its cluster C_i ;
- 6 **if** *popularity* $> \Delta$ **then**
- 7 | Perform a two-hop search;
- 8 **else**
- 9 | Perform a search based on selectivity;
- // High-selectivity and negatively correlated [14].
- 10 Select the Top- K clusters with the highest attribute frequency;
- 11 Choose entry points for these clusters;
- 12 Graph traversal from these entry points;

the estimated count. The size and accuracy of CMS are governed by the relative error (ϵ) and the failure probability (δ) [24].

CDF-based prediction model: We employ a CDF-based regression model to estimate the frequency difference between ranges. Kraska et al. [25] introduced *learned indexes* to predict the memory location of a specific key using a learned model instead of a complex data structure. Initially, it requires a training phase, during which a regression model is trained using the cumulative distribution function (CDF) of the items, along with the sorted items themselves. Learned model predicts the corresponding CDF value for a new attribute value, which can then be used for various purposes such as locating memory positions, estimating item frequency, and performing approximate range filtering [26], [27], [28].

K -minwise hashing: In this study, we employ K -minwise hashing to estimate the intersection of data items across overlapping clusters in a PG (clustering details are in Section III-B). This data structure is built on top of CMS or CDF-based structures, where each cell or a specific CDF range (e.g., 0.0–0.1) stores a set of hash values computed from the data identifiers using a hash function $h(f)$ [29]. The accuracy of these estimations depends on the fingerprint size and the total number of fingerprints. Following OmniSketch [29], we apply K -minwise estimation to compute the intersections of keys between clusters in Section IV.

III. IN-MEMORY PREDICTORS FOR HYBRID SEARCH

In this section, we discuss the construction of in-memory predictors for hybrid search.

A. Solution Overview

The first step is to construct the HNSW graph for the dataset, followed by cluster formation. Layer 0 of the HNSW is known to be dense, with all data vectors interconnected [7], [13]. Starting from the first node (i.e., the memory location of the first data item) in layer 0, we select its nearest neighbors

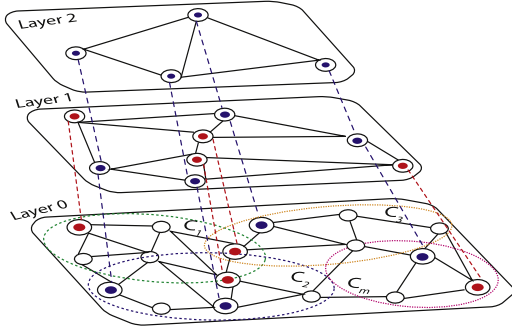


Fig. 3. Clustered HNSW: layer 0 shows overlapping nodes.

(two-hop neighbors) to form a cluster. Subsequently, we choose the next unvisited node to construct the next cluster and repeat the process iteratively. Each cluster contains in-memory predictive data structures. Details of cluster construction are provided in Section III-B.

For point queries, we employ the CMS as an in-memory predictor (predictor or sketches are used interchangeably), whereas for range filtering, we use a learned CDF-based regression model. Furthermore, we introduce a novel sketch type for estimating the popularity of conjunctive queries. We provide a detailed discussion on different low selectivity queries execution in Section IV. During post-filtering of nodes in the HNSW graph, we decide whether to expand to two-hop neighbors for each node based on the popularity threshold of query attributes within the corresponding node cluster. Moreover, for highly selective queries, we leverage the top- k clusters with the highest frequency of the query attribute, guided by effective entry points (index positions) in layer 0 determined during cluster construction time as detailed in Section IV-D. Alongside predictive algorithms, we also consider various optimizations for memory usage and disk access as detailed in Section IV. Algorithm 1 outlines the entire procedure for the proposed predictor-based approach.

B. Clusters Construction

For popularity-aware hybrid search, layer 0 of the HNSW graph is divided into several clusters, as shown in Fig. 3. To create a cluster, a node n_i (starting node n_0) is selected as the entry point for traversal. Initially, the in-memory sketches are updated with the attributes of n_i : the CMS for point queries and a CDF-based regression model for range queries. The choice between a CMS and a CDF-based regression model depends on the attribute type. For example, numerical attributes such as age or height, which naturally support range queries, use a CDF-based model. In contrast, categorical attributes such as disease are maintained using a CMS. After this update, the two-hop nearest neighbors of n_i are visited, and their attributes are also used to update the in-memory sketches.

We employ two different strategies for clustering, depending on the value of M in the HNSW graph. For smaller values of M (e.g., 20, 30, and 40), we typically expand from a given node to its neighbors beyond two hops, continuing until the cluster reaches a specified size C_s . In contrast, for larger values of M , we only

rely on the two-hop neighbors of the current node for sketch updates, without further expansion—this is the most common setting in our experiments. In each iteration, we select a new unvisited node as the next starting point and repeat the process of its two-hop neighbors expansion; moreover, clusters may have overlapping nodes. In-memory sketches are stored as tuples of the form $\langle C_i, E_i, \text{sketches} \rangle$, where C_i denotes the cluster identifier and E_i represents index positions (i.e., starting nodes for cluster traversal), which later act as entry points for high selectivity queries. This process proceeds incrementally by selecting unvisited nodes from the HNSW graph to initiate new clusters.

CMS update: Each cluster maintains its own CMS of fixed size. The parameters w (width) and h_f (number of hash functions) are chosen based on the desired error bound ε . Typically, we employ a small-sized CMS to minimize memory usage. For each node n_i , the categorical attribute is hashed using the CMS hash functions, and the corresponding counters at the computed positions in each of the h_f rows are incremented accordingly. Once the cluster reaches the threshold, the CMS is saved into the predictor data structure, and a new CMS instance is initialized.

CDF-based regression model update: During cluster construction, attributes are initially stored in a temporary data structure. Once a predefined cluster threshold is reached, the data structure is sorted. After sorting, the CDF is computed for each attribute value. A linear or piecewise linear regression model is then trained using the computed CDF and the corresponding attribute values [25], [26]. The number of regression models is proportional to the number of clusters, with each model defined by its slopes and intercepts. We primarily adopt a linear regression model, as used in learned indexes [25], to ensure efficient prediction. However, our approach can be extended to incorporate piecewise linear regression for more accurate CDF estimation [26], at the cost of a small overhead for segment identification during clustering and prediction.

1) Time Complexity: The base iteration involves processing all level 0 nodes, resulting in a time complexity of $O(n)$. However, two hops are performed only on selective nodes, representing $\alpha \times D$, where α is the selective nodes and D denotes the degree of the two hops for the selective node n_i . This D can be computed as $D = d + \sum_{i=0}^d d_i$, where d is the degree of the node n_i , and d_i represents the i^{th} one-hop neighbor for node n_i . Typically, d for the node n_i depends on the factor M of HNSW.

After considering this, the total time complexity for generating the clusters is $O(n + \alpha \times D)$. While the time complexity for updating the in-memory sketch, such as the CMS, is constant, however, the CDF-based regression model includes sorting attributes that form a subset of the original dataset. In the worst case, this sorting operation can take $O(n \log n)$.

2) Memory Analysis: Each cluster utilizes in-memory sketches for its attributes. For CMS, we use size $\sum_{i=0}^{|C|} (h \times w)$, where h represents the number of hash functions and w is the width of the CMS. Similarly, the CDF-based regression model maintains only the slope and intercept values for each cluster.

Another data structure is utilized to maintain the relationship between the HNSW nodes and the cluster id . A node n_i can belong to multiple clusters. The space usage across all clusters

Algorithm 2: Point Filtering.

Input: Query vector q_v , query attribute q_l , cluster to mappings (clusters), and popularity threshold (Δ) for the q_p

Output: Top- K nearest neighbors for q

```

1 Initialize  $R_{distPQ} \leftarrow \emptyset$ ; // initialize empty result set
2 Top- $K$ _Nodes  $\leftarrow$  HNSW-PF( $q_v$ );
3 while Top- $K$ _Nodes is not empty do
4    $n_i \leftarrow$  Top- $K$ _Nodes.First();
5   if attribute of  $n_i$  equals  $q_l$  then
6      $R_{distPQ}.add(n_i)$ ;
7   // Find the cluster ID for  $n_i$ 
8   clusterID  $\leftarrow$  clusters( $n_i$ );
9   popularity  $q_l \leftarrow \frac{CMS_{clusterID}(q_l)}{N_{clusterID}}$ ;
10  if popularity  $q_l > \Delta$  then
11    TwoHopNeighbors  $\leftarrow$  TwoHopSearch( $n_i$ );
12    Add neighbors matching  $q_l$  to  $R_{distPQ}$ ;
13  TopKNodes.pop();
14 return  $R_{distPQ}$ 

```

can be expressed as $O(n \sum_{i=0}^m c_\beta)$, where the c_β represents the number of clusters for the nodes identifier.

Bit-level encoding of cluster membership: To reduce cluster-node relationship space usage, we propose a strategy that utilizes a fixed-size bit array based on the size of the dataset. We use fixed-size fingerprints—compact bit arrays such as 8-bit, 16-bit, or 24-bit—to store the cluster identifier for each node in layer 0 in the HNSW graph. All bits are initially set to 0. The total size of the bit array is given by $|S| \times \text{Fingerprint size}$. We empirically evaluated various cluster sizes, and our results show that a 16-bit fingerprint is sufficient for datasets of up to 1 million entries (65,536 distinct clusters). Similarly, a 24-bit fingerprint can accommodate up to 16.8 million clusters.

IV. HYBRID QUERY SEARCH

In this section, we discuss query operators (high- and low-selectivity queries) and several optimizations.

A. Point Filtering

For point predicate filtering, we utilize the CMS algorithm to estimate the popularity of the query attribute. Initially, the top- K nearest vectors to the query vector are retrieved using a standard HNSW graph. From the top- K nodes, a subset is chosen for a two-hop search based on the CMS prediction. Algorithm 2 describes the complete procedure of hybrid point query predicate evaluation.

The algorithm takes as input a query (consisting of a vector and its associated attributes), a cluster array that maps each node to its corresponding cluster, and a popularity threshold. The output is the top- K nearest vectors to the query q that also satisfy the point predicate filtering condition.

Initially, the algorithm initializes an empty result set. It then applies HNSW post-filtering to the query vector, retrieving the top- K nearest neighbors (line 2). It proceeds by iterating over each of these top- K nodes. For every node n_i , the algorithm checks whether its attribute matches the query attribute q_p , and if so, it adds the node to the result set (lines 4–6). Next, the algorithm identifies the cluster associated with node n_i (line

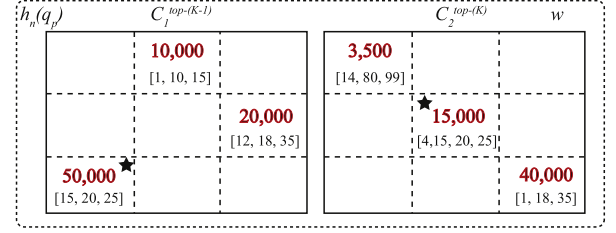


Fig. 4. CMS with K-minwise hashing.

7). It then estimates the popularity of the query attribute within that cluster using the in-memory CMS data structure (line 8). If the popularity exceeds the specified threshold (Δ), a two-hop nearest neighbor search is performed from node n_i . During this search, all candidate neighbors are evaluated against the query predicate q_l , and qualifying results are added to the result priority queue.

1) Search Optimization Using K-Minwise Hashing: Our clustering approach shows that a single node can belong to multiple clusters (see Section III-B). To prevent the need for a two-hop search from each top- K node, we utilize a k -minwise hashing strategy. This method involves maintaining the fingerprints of k identifiers for smaller hash values within each CMS cell during clustering.

Consider an example based on Fig. 4. After post-filtering, the node ranked top- $(K - 1)$ selects cluster C1. In this cluster, the corresponding predicate cell has a count of 50,000. Additionally, the cell stores k fingerprints representing the identifiers of the nodes that have updated it. Similarly, the top- K node selects cluster C2, where the count in the corresponding cell is 15,000. According to the popularity threshold, it would ideally perform a two-hop search. However, intersection (75%) size analysis suggests that this is unnecessary, as most of its nodes have already been visited through the two-hop search executed for the top- $(K - 1)$ node.

2) Time Complexity: The time complexity for finding the top- K nearest items after post-filtering in HNSW is $O(\log N)$. Determining the cluster and the attribute's popularity using the CMS is performed in constant time, $O(1)$. Let $D = d + \sum_{i=0}^d d_i$, where D represents the total number of two-hop neighbors for any layer 0 node in HNSW, as described earlier. Traversing each of the top- K items up to two hops requires $O(K \times D)$ time. Our optimization strategy reduces this to $O((K - y) \times D)$, where y denotes the number of clusters excluded from two-hop exploration. Additionally, we employ a red-black tree to store identifier fingerprints, allowing intersection computation in $O(B \log B)$, where B is the number of items in each CMS cell.

B. Range Filtering

In this study, we utilize a CDF-based regression model to determine the frequency of items within specified query ranges. The process of range filtering is akin to point filtering, such as identifying the top- K items and their relevant clusters. However, instead of using a categorical attribute in the query, the range query consists of two numerical values. In this case, we have a regression model that predicts the CDF values for the query

TABLE III
HIGH-ORDER TRAVERSAL CONDITIONS

Query Predicate	CMS	CDF	Condition
$q = \text{attr}; q \neq \text{attr}$	✓	×	$\text{CMS}(\text{attr}) > \Delta; 1 - \text{CMS}(\text{attr}) > \Delta$
$q \in [\text{attr}_1, \text{attr}_2]$	×	✓	$\text{CDF}(\text{attr}_2) - \text{CDF}(\text{attr}_1) > \Delta$
$q > \text{attr}_1; q < \text{attr}_1$	×	✓	$1 - \text{CDF}(\text{attr}_1) > \Delta; \text{CDF}(\text{attr}_1) > \Delta$
$q = a \wedge (q \geq a_1 \wedge q \leq a_2)$	✓	✓	$\text{IntersectionSize}(\text{Kminwise}) > \Delta$
$q = a \vee (q_1 \geq a_1 \wedge q_2 \leq a_2)$	✓	✓	$\text{CMS} > \Delta \vee \text{CDF}_{\text{range}} > \Delta$
$q = a_1 \vee a_2 \dots \vee a_n$	✓	✓	$\exists i : \text{cond}(a_i) > \Delta$

ranges. Ultimately, the difference in CDF values serves as an estimate of frequency or threshold, guiding us in a two-hop search from the identified top- K nodes.

C. Low-Selectivity Querying With CMS–CDF Guided Traversal

We also consider queries involving conjunctive operators that combine both point and range filtering. To handle such queries, we employ a CMS and CDF-based model for the relevant cluster. However, these models only predict frequencies for attributes or provide the CDF for range queries.

To bridge this gap, we leverage the intersection between CMS cells and the regression model. Each CMS cell stores K fingerprints of identifiers (IDs), but the regression model lacks corresponding fingerprints. To address this, we sample fingerprints for identifiers across different CDF value ranges within each cluster. For example, in the CDF range from 0 to 0.1, we might have sampled IDs $\{1, 18, 19, 2000\}$. This approach allows us to maintain K fingerprints for CDF models.

During inference, we first locate the CMS cell corresponding to the node’s cluster. Next, we compute the CDF for the query-specified range. Finally, we intersect the fingerprints from the CMS cell with those associated with the corresponding CDF range. The size of this intersection informs the two-hop search decision.

In-memory sketches can also be applied to a wide range of low-selectivity queries, as listed in Table III. However, for each case, the condition for higher-order traversal is modified, as also detailed in Table III. For example, a query that contains multiple OR conditions across different attributes is considered a low-selectivity query, since a large portion of the dataset is likely to satisfy the overall query predicate. In such cases, we simultaneously observe the frequencies of the attributes and select a two-hop traversal if any attribute condition exceeds the popularity threshold (Δ).

Popularity threshold: To compute the popularity threshold (Δ), we rely on the query selectivity. For point queries, attribute selectivities are stored in a map, whereas for range queries, we employ an equi-width methodology in which the query selectivity is estimated from the relevant bins covering the query range. This enables efficient computation during the search. We empirically tested different combinations of query selectivities to examine the trade-off between time and recall in Section V-C. Our results indicate that choosing a threshold smaller than the query selectivity—for instance, 0.3 for a query selectivity of 0.8—incurs high overhead in distance computations. In contrast, selecting the threshold within a small factor of the query selectivity (e.g., $0.8 \pm \gamma$, where $\gamma \in [0.1, 0.2]$) provides a better

trade-off between computation time and recall. We adopt this factor for the (Δ) selection.

For queries with unknown selectivity, we propose a dynamic approach that leverages cluster information along with the normalized minimum (lower bound) and maximum (upper bound) values fetched from all in-memory predictors (computed offline) and search parameters of the query. The popularity threshold (Δ) is computed as; $\Delta = \text{lower bound} + \alpha \cdot (\text{upper bound} - \text{lower bound})$, where α is a normalization factor. In practice, α is computed as $\alpha = \frac{efs}{efs_{\max}}$, where efs is the search parameter for a query and $efs_{\max}$ is determined from the graph construction parameter M and is typically set to $s \times M$ in HNSW, with s being a small natural number (e.g., $s = 2$) or the maximum allowable efs value. Smaller values of efs result in smaller threshold values, whereas larger values of efs yield larger values for Δ , reflecting that a larger portion of the graph has already been explored during post-filtering.

D. High Selectivity and Negatively Correlated Queries

We also address high-selectivity queries, where the query attribute is highly selective and distant from its nearest neighbors (i.e., negative correlation [14]). Although pre-filtering is a common approach [14], however, we propose an alternative that utilizes cluster-level information to guide graph traversal.

Specifically, we select the top- k clusters in which the query attribute has the highest frequency. This process can be easily parallelized. Each cluster maintains index positions (i.e., entry points at layer 0 of the HNSW graph). From all of these positions, we perform a two-hop traversal to identify the top- K items that also satisfy the query predicate.

E. Disk Access Optimization

Typically, the attribute values of vectors are stored on a disk or in the cloud. Accessing these locations can be expensive, especially when searching for whether an item exists. To address this issue, we propose a hash-based two-way filter that is similar to a bloom filter. Initially, we create a bit array of size S that is used during cluster construction. This bit array is then equally partitioned into k sized fingerprints. During the clustering process, we use h hash functions to hash the file identifier. The output of these hash functions directs us to the respective partitions associated with the designated fingerprint.

Let’s assume that id is the identifier for the file that contains the attributes of vectors. Initially, we use hash functions $hf(id)$ on id . The outputs of the hash functions are 1, 22, and 100. We use these outputs to update their respective bits in the fingerprints. Next, we apply the same hash function to the attribute l . The result of $h(p) \bmod \text{fingerprints_size}$ is 2, so we turn on the second bit of the fingerprint, which has a size of 8. During query processing, the same procedure is performed. If all corresponding bits are set in the identified fingerprint, we proceed to search within the file; otherwise, the file for search.

V. EXPERIMENTAL SETUP AND RESULTS

This section presents an experimental evaluation of the proposed approach. Typically, we focus on low selectivity queries

TABLE IV
DATASETS PROPERTIES

Dataset	Domain	# Vectors	# Queries	Vector Dim	Query type	Avg. Query Selectivity	Structured Data
Sift1M [30]	Images	1,000,000	10,000	128	Point	0.083	Random int (1–12)
Paper [12]	Academic papers	2,029,996	10,000	200	Point	0.083	Random int (1–12)
BEIR [31]	Text documents (IR corpus)	1,463,690	1,160	1024	Point	0.28	Source dataset
TripClick [32]	Text documents (Medical field)	1,142,058	1,000	768	Point	0.26	Clinical area
					Range	0.39	Document date
YT-Rgb [33]	YouTube videos	1,289,619	1,500	1024	Point	0.11	Genre
					Range	0.33 (Likes), 0.35 (Views)	Likes, Views
YT-Audio [33]	YouTube audio	1,289,619	1,500	128	Range	0.33 (Likes), 0.35 (Views)	Likes, Views

because for high selectivity queries, pre-filtering ensures perfect recall. However, we also provide an experimental evaluation for the novel PG-based solution designed for high selectivity hybrid queries.

A. Datasets and System Configuration

We use five real-world datasets from diverse domains with varying characteristics. Moreover, we construct query sets by sampling a subset from each dataset. Table IV provides detailed descriptions of the datasets.

Hardware and software: All experiments were conducted on two machines: one equipped with four Intel Xeon E5-2697 CPUs at 2.40 GHz (72 cores) with 216 GB of RAM, running Ubuntu 18.04, and another with an AMD EPYC 9654 CPU (96 cores) and 660 GB of RAM, running Red Hat Enterprise Linux 8.10. All comparative experiments were performed on the same hardware to ensure fairness. The proposed predictive approach is built upon HNSWLib,¹ with each experiment conducted using 40 threads in a C++ environment.

B. Comparison Methods

Our main comparison strategy is divided into three main components. First, we evaluate the proposed predictor-based approach (Predictor)² against several methods, using the construction parameters reported in their original studies. For the predictor, we adopt parameter settings consistent with ACORN [14], using $M = 2M$. We also include the original implementations of all baselines for a fair comparison. We choose the Δ empirically based on selectivity.

Second, we examine various extensions of HNSW specifically designed for hybrid queries. We set the parameters $M = 256$ and $efc = 600$, where M controls graph connectivity and efc determines the search depth during construction, respectively. These relatively high values are chosen to ensure high recall across all methods. In this case, we choose the Δ empirically based on selectivity.

Third, we compare the most recent agnostic PG-based techniques, including NaviX [13], ACORN-1 [14], HNSW-PF, a dynamic variant of the predictor, and our proposed predictor. All methods are evaluated using dataset-specific parameters recommended by ACORN [14]. To ensure a fair comparison, we fix the

construction parameter to its base value M (i.e., without scaling to $2M$) and conduct all experiments under identical hardware settings. Moreover, we rely on the dynamic computation of the Δ for the predictor approach. We vary efc from 20 to 3000, with increments of 100 and 200. Additionally, we compare the in-memory predictor against HNSW-based extensions using ACORN-recommended parameters, while keeping M fixed across all schemes.

We provide a comparison of HNSW-based techniques across different query selectivity and correlation levels, focusing on relevant methods for predictor-based approaches and restricting datasets by dimensionality (low, medium, high).

ACORN: ACORN³ utilizes the FAISS library for hybrid vector search. ACORN evaluates the predicate condition during graph traversal (in-filtering). In our initial experiments, we applied this implementation to both ACORN-1 and ACORN- γ . Subsequently, we focused exclusively on ACORN-1 as our primary baseline. For $M = 256$ and $efc = 600$, we used HNSWLib for ACORN-1 for a fair comparison with the densest graph.

NaviX: NaviX is most recent hybrid search methodology also uses the FAISS library.⁴ It relies on in-filtering, where each query determines high-order graph traversal based on the online computation of the query's local selectivity.

Dynamic predictor: Similar to ACORN and NaviX, the *dynamic predictor* is an extension of the predictor approach. It also relies on in-filtering, where the choice of one- or two-hop search is determined by the current popularity of the attribute.

HNSW Post-filtering (HNSW-PF): We consider HNSWLib for post-filtering. Rather than focusing solely on the top- K items, we consider the efc number of nodes as candidate nodes and then filter for the top- K items.

Two-hop post-filtering (Two-hop PF): We provide a variant of the two-hop extension that utilizes the top- K filtered nodes through a post-filtering approach. This method applies the two-hop strategy only to the top- K nodes.

IRange Graph: IRange Graph is built on top of HNSWLib⁵ for range queries. This filter initially requires sorting of the data items. Furthermore, queries require the index position of their respective ranges. We propose finding the index online and using a binary search for efficient identification.

¹<https://github.com/nmslib/hnswlib>

²https://github.com/AdeelAslamUnimore/Clustered_Hybrid_Search

³<https://github.com/guestrin-lab/ACORN>

⁴<https://github.com/gaurav8297/faiss-navix>

⁵<https://github.com/YuexuanXu7/iRangeGraph>

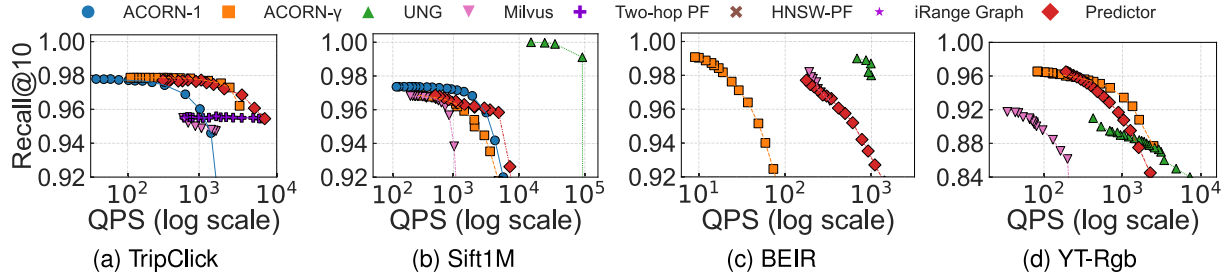


Fig. 5. QPS vs. Recall curves for point queries using the original study parameters (y-axis label in (a) is shared across figures).

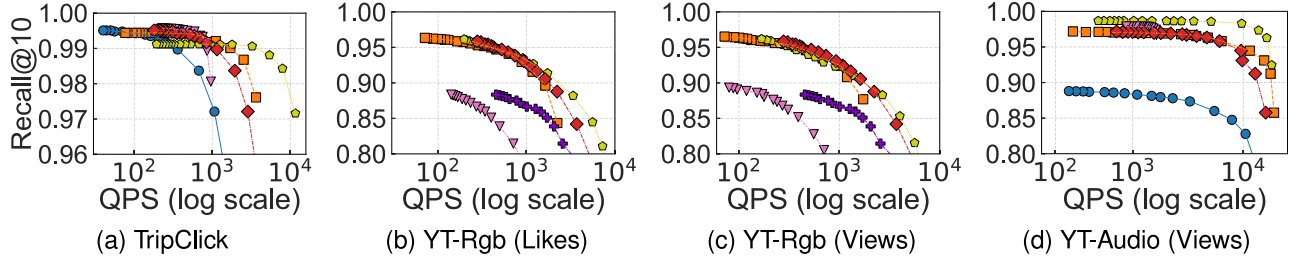


Fig. 6. QPS vs Recall curve for range queries with original studies' parameters.

Unified Navigating Graph: UNG⁶ is designed for point query predicates. Initially, it needs to convert the vector labels into integers. For optimal performance, it is advisable to assign lower integer IDs to the most frequent labels. This requires a pre-processing stage to determine label frequencies and assign an integer ID to each label. Additionally, the query dataset must include a file containing the label dataset.

Milvus: Milvus is an open source vector database that also supports hybrid vector search. We opt for its original implementation with the C++ SDK.⁷

C. Evaluation of Hybrid Queries Across Search Strategies

Note that the y-scale (Recall@10) of different charts varies among all datasets; we have chosen the best one to highlight the differences between the evaluated methods.

Baseline comparison with original configurations: Fig. 5(a)–(d) display the results of point-predicate filtering. The results show that the predictor-based agnostic approach significantly outperforms other naïve HNSW-based methods, such as ACORN-1 and two-hop PF. Furthermore, its performance is nearly similar to that of dedicated search techniques such as ACORN-γ and UNG. For the TripClick dataset, each entry consists of a list of metadata, which is generally treated as a containment search. In this context, UNG does not achieve a recall of 96% as depicted by Fig. 5(a).

In experiments on datasets with small embedding sizes or a single attribute value (Sift and BEIR), the predictor-based approach achieves performance comparable to ACORN-γ, as illustrated in Fig. 5(b) and (c). In contrast, the dedicated graph UNG demonstrates superior performance in these settings. However, UNG exhibits very low recall on the YouTube video dataset

due to its high embedding dimensionality and complex attribute structure (e.g., multiple genres per video), as shown in Fig. 5(d). Although UNG achieves slightly better performance than the proposed approach in some cases, it is not agnostic to predicates and requires several preprocessing steps for graph construction. In contrast, our predictor-based approach remains more flexible and agnostic, relying solely on the default HNSW for search.

Fig. 6(a)–(d) present the results of range filter queries. In this case, the IRange Graph is considered a more recent and dedicated graph for range searches. This type of structure requires a pre-processing step for sorting the entire data structure. The results indicate that this dedicated graph query processing performs well only when the size of the embedding is small. However, in larger settings, the proposed predictor approach demonstrates comparable performance and may even outperform the alternative solution.

Analysis of HNSW-based hybrid query methods with larger construction parameter: Figs. 7 and 8 present a comparative analysis of the QPS-recall tradeoff for naïve HNSW-based schemes with fixed parameter sizes ($M = 256$ and $efc = 600$). In this experiment, we analyze three different versions of the proposed predictor approach. The first version, called the predictor (Max), examines all clusters associated with a specific attribute and selects the cluster with the highest prediction value. The second version, predictor (random), randomly selects a cluster. Lastly, the predictor (intersection: using k-minwise hashing) focuses on the intersection of keys.

Fig. 7(a) presents the results for the TripClick dataset for point predicate evaluation. The ACORN-1 achieves a 99% recall for 150 QPS. However, HNSW-PF is unable to reach the 99% recall for 3200 $efcs$. Similarly, the two-hop PF based on $efcs$ also achieves a 99% recall with 493 QPS. The proposed predictor-based approach, along with its optimized version, demonstrates query processing performance that is 3x and 3.8x superior to

⁶<https://github.com/YZ-Cai/Unified-Navigating-Graph>

⁷<https://github.com/milvus-io/milvus-sdk-cpp>

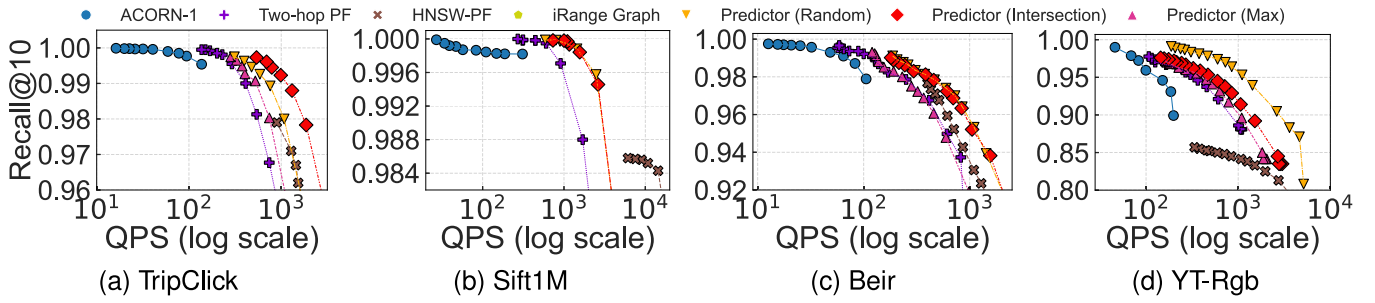


Fig. 7. QPS vs Recall curve for point (the y-axis label in (a) is shared across figures).

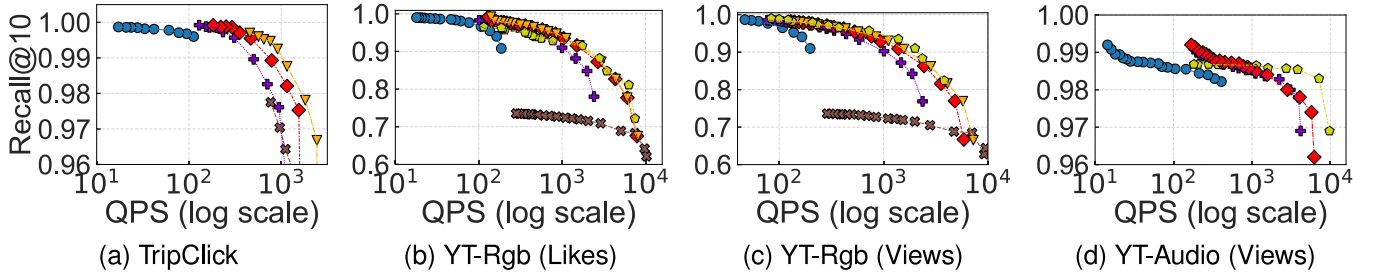


Fig. 8. QPS vs Recall curve for range queries. Note that the IRange graph method is available only on numeric attributes.

ACORN-1, respectively. Additionally, it is also 1.3x and 1.8x higher than the two-hop PF approach.

Similarly, Fig. 7(b) illustrate the QPS–recall curves for the SIFT datasets; as the paper dataset reports similar performance, we omit its corresponding graph. We observed that the intersection-based predictor has almost 6x and 1.5x better QPS for higher recall than ACORN-1 and two-hop PF. Fig. 7(c) and (d) illustrate the results of point-query evaluations on the BEIR and YouTube datasets. The results show that the intersection-based predictor performs significantly better, achieving speedups of $3\times$ and $4.4\times$ over ACORN-1, and $2\times$ and $2.1\times$ over the two-hop PF method at 99% recall.

Fig. 8(a) illustrates the results of *range queries* on the TripClick dataset. The ACORN-1 achieves 112 QPS with a 99% recall rate. In contrast, the HNSW-PF approach demonstrates similar shortcomings to point queries within the same dataset, failing to reach a 99% recall for an 800 *efs* value. The two-hop PF search process achieves 310 QPS to attain a 99% recall. The CDF-based predictive approach performs 3.3x better than ACORN-1 and 1.3x better than the two-hop search. Similarly, a randomly chosen cluster shows an 8.2x and 3x better performance compared to the alternatives.

Fig. 8(b) and (c) present the *range query* results for the YouTube dataset, focusing on video embeddings and using *likes* and *views* as range attributes. Our predictive approach achieves a $2\times$ improvement over ACORN-1 and a $1.4\times$ improvement over the IRange Graph for the *likes* attribute. The optimized version of the predictor performs better, showing a $3.2\times$ improvement over ACORN-1. Similarly, better performance is observed for the *views* range attribute.

Fig. 8(d) illustrates the results for YouTube data using audio embeddings and a *range predicate* on the *view* data field. It shows that ACORN-1 achieves a recall of 99% with 16 QPS,

while the HNSW-PF attains a recall of 74% for 3000 *efs*. The two-hop PF variant achieves 144 QPS with a recall of 99%. Similarly, IRange Graph recall remains at 98% for 3000 *efs*. The proposed prediction-based method reaches a recall of 99% with 237 QPS. Similar performance improvement is observed for the same embeddings where the predicate is based on the *like* attribute; therefore, we omit the corresponding graph.

ACORN-1 performs a two-hop search from each node, leading to increased processing time. The HNSW-PF method focuses on nearest neighbors but often misses neighbors meeting predicate constraints. Additionally, a two-hop PF-based search using *efs* increases query latency by exploring many candidates. The IRange filter, designed for range queries, requires reconstructing the original graph and sorting for each attribute. In contrast, the proposed predictive approach identifies nearest neighbors and selectively explores hops where filtered item popularity exceeds a threshold, improving recall efficiency and reducing hops.

Unified evaluation of agnostic PG-based techniques: Fig. 9 presents the results of the hybrid queries for HNSW-based predicate-agnostic methods under the unified setting reported in ACORN [14] (i.e., without scaling to 2M for ACORN-1). Moreover, we include the latest NaviX [13] and our proposed dynamic in-filtering predictor. Fig. 9(a) shows that the predictor achieves a $1.7\times$ – $2.6\times$ speedup over the dynamic predictor while improving recall by 7–10% across varying *efs*. Similarly, compared to NaviX and ACORN-1, the predictor achieves approximately $3\times$ and 4 – $5\times$ higher QPS, respectively, while maintaining slightly higher recall. Fig. 9(b) presents results for the Paper dataset. The predictor improves recall by 12–13% compared to the dynamic predictor while maintaining comparable QPS. In comparison with NaviX, it is $1.2\times$ – $1.6\times$ faster and achieves approximately 9–10% higher recall. Compared to ACORN-1, it achieves

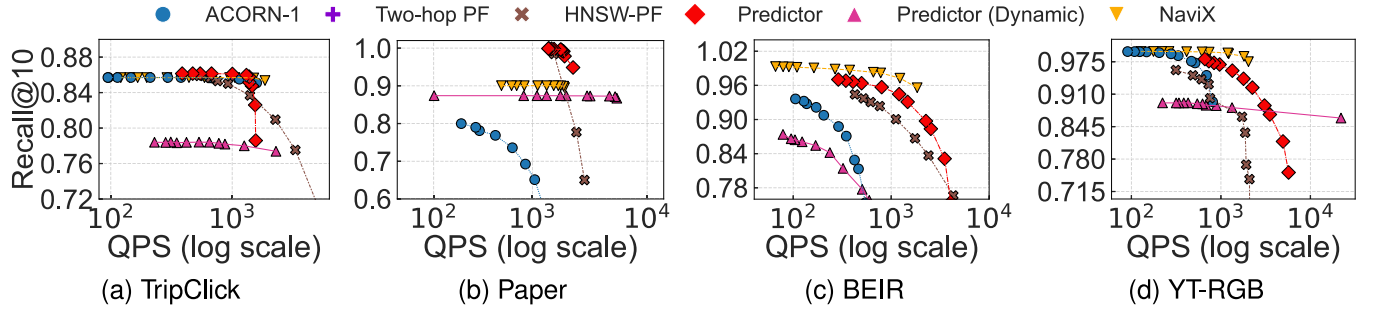


Fig. 9. QPS vs Recall curve for HNSW-based schemes (the y-axis label in (a) is shared across figures).

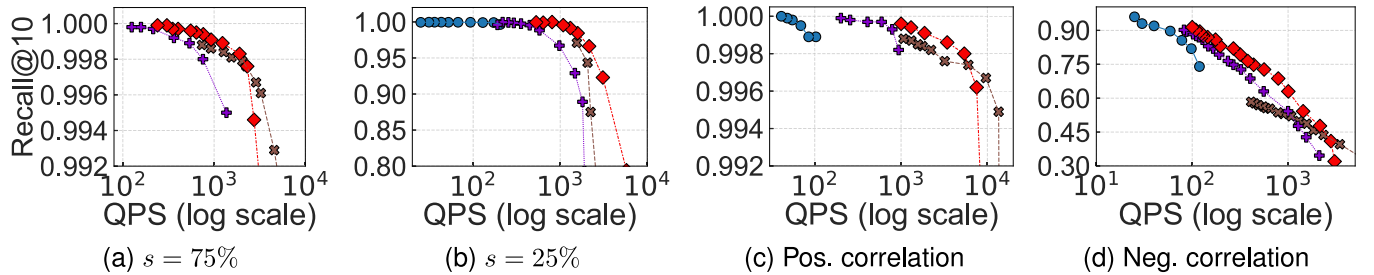


Fig. 10. Varying selectivity and correlation queries (the y-axis label in (a) is shared across figures).

$1.6 \times -2.6 \times$ higher QPS with significantly improved recall of 20–30%.

Fig. 9(c) and (d) show the results for the BEIR and YouTube-RGB datasets. The proposed predictive approach achieves $2.5 \times -3.3 \times$ higher QPS than the dynamic predictor while improving recall by 5–8%. Compared to NaviX, the predictor provides $3 \times -4 \times$ higher QPS, with only a slight reduction in recall. Furthermore, compared to ACORN-1, the predictor is $6 \times -7 \times$ faster, with only a small reduction in recall. The proposed predictor relies on post-filtered nodes; however, in-filtering techniques (ACORN, NaviX, and Predictor (dynamic)) evaluate predicates during graph traversal, which can either lead to premature convergence in low-selectivity queries or incur significant overhead due to online heuristic computation (local-selectivity computation [13]) at each node.

Queries selectivity and correlation: Fig. 10 presents results under varying selectivity and correlation levels between queries and the dataset. We use high-dimensional embeddings from the YouTube-RGB and TripClick datasets, with construction parameters $M = 256$ and $efc = 600$, and synthesize attribute values based on the desired selectivity and correlation.

With a selectivity of 75%, an attribute value is assigned to 75% of the dataset records. For correlation, we assess the distance between query vectors and the dataset. For negative correlation, we select the top 100 records with the greatest distance, while for positive correlation, we choose the top 100 nearest vectors. For average correlation, attributes are assigned uniformly. We only compare HNSW-based graph approaches.

Fig. 10(a) and (b) show results across varying selectivity levels. At 75% selectivity, ACORN-1 does not outperform the other HNSW-based schemes, while the predictive approach achieves a 99% recall with better QPS compared to two-hop PF and

HNSW-PF. For 25% selectivity, ACORN-1 performs similarly, but HNSW-PF recall drops to 97%.

Fig. 10(c) and (d) present results under positive and negative query–dataset correlations. For positive correlation, HNSW-PF improves performance but fails to reach full recall, whereas the proposed predictor achieves 99.9% recall with 839 QPS using a one-hop search. In contrast, under negative correlation, HNSW-PF remains suboptimal, failing to reach 90% recall, while ACORN-1 achieves higher recall. The proposed approach attains recall comparable to ACORN-1, while delivering a $3 \times$ improvement in QPS.

Selectivity analysis under varying query and data distributions: To evaluate query selectivity, we analyze both query and dataset selectivity using the larger Paper dataset with parameters $M = 32$ and $efc = 64$. We compare the performance against agnostic PG-based in-filtering techniques, as illustrated in Fig. 11. We control selectivity by adjusting the attribute distributions in both the queries and the data. For instance, in the 80-20 setting, 80% of the queries have an 80% selectivity, meaning that 80% of the data satisfies the predicate, while the remaining 20% follows a normal distribution.

Fig. 11(a)–(d) present results across different query–data distributions (80–20, 60–40, 40–60, and 20–80). Across all settings, the predictor consistently achieves higher recall than other methods while improving QPS. Specifically, it provides a $1.1 \times -1.35 \times$ speedup for the 80–20 distribution (Fig. 11(a)), $1.15 \times -1.29 \times$ for 60–40 (Fig. 11(b)), approximately $1.3 \times$ for 40–60 (Fig. 11(c)), and $1.3 \times -1.7 \times$ for 20–80 (Fig. 11(d)).

The proposed predictive approach employs post-filtering with a two-hop search, focusing on attribute popularity in nearest-neighbor clusters. For low-selectivity queries, it initially narrows candidates through nearest neighbors, followed by a two-hop

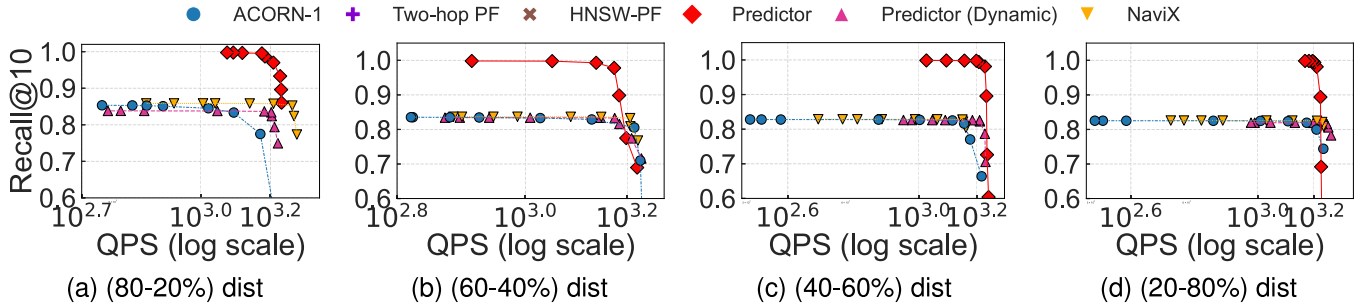


Fig. 11. QPS-Recall performance under varying query and data selectivity (y-axis label in (a) is shared across figures).

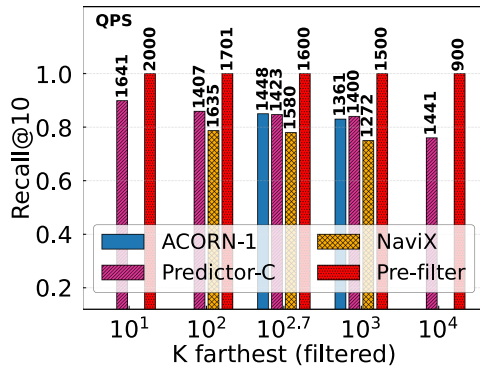


Fig. 12. High selectivity queries with negative correlation.

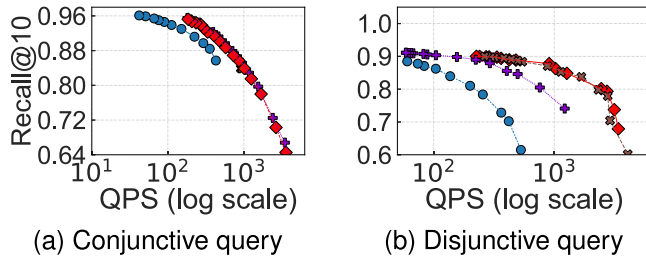


Fig. 13. Conjunctive and disjunctive queries (y-label in (a)).

selection after filtering. In contrast, other methods use intraversal filtering, which can lead the search towards distant nodes in low-selectivity scenarios, delaying convergence and risking missed top-k results.

Fig. 12 shows results for high-selectivity queries with negative correlation, where the x-axis represents the number of farthest data items satisfying the query predicate for $efs = 80$. The pre-filtering method generally outperforms others, while our predictor approach, which effectively leverages cluster-level information and chooses entry points, also achieves higher recall and improved QPS. For 100 to 500 farthest data items, ACORN-1 and NaviX perform well; however, our clustering technique consistently delivers better recall and QPS. At $K = 10,000$, our predictive method maintains high recall by accurately identifying relevant entry points and exploring nearest neighbors. In contrast, ACORN-1 and NaviX struggle to find suitable entry points, often converging on irrelevant nodes, resulting in a significant drop in recall.

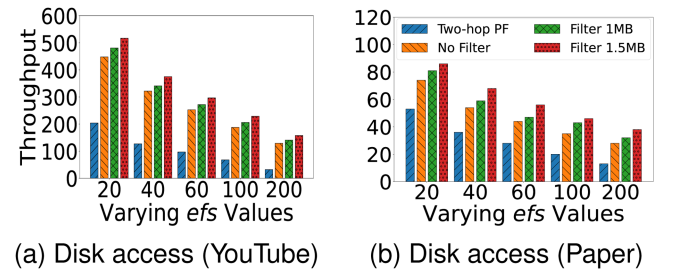


Fig. 14. Disk access (y-label in (a); legend in (b)).

Conjunctive and disjunctive queries evaluation: Fig. 13 presents the results of conjunctive and disjunctive queries (combining range and point filters within a single query) on the YouTube video datasets. For the conjunctive query, none of the strategies reach 99% recall. ACORN-1 achieves 96% recall at 42 QPS, whereas the two-hop PF method reaches the same recall at 219 QPS. The predictive approach slightly improves performance to 235 QPS, as illustrated in Fig. 13(a). Restricting exploration to two-hop neighbors, particularly when the combined frequency of the identified attribute is high, enables higher throughput and improved recall.

Similarly, for disjunctive queries, most queries have low selectivity. In this case, as the number of records satisfying the queries increases, the post-filtering process performs better in terms of accuracy and efficiency. However, our predictor approach also performs comparably and in some aspects better than the HNSW post-filtering method. Additionally, it outperforms the ACORN-1 approach as depicted by Fig. 13(b).

Disk optimization using filter: Fig. 14 illustrates the impact of the proposed two-way filter on disk access performance, compared against the two-hop PF method, across the YouTube video dataset and the Paper dataset.

Fig. 14(a) shows the results for the YouTube video. The two-hop PF method achieves 204 QPS at $efs = 20$, while the predicate-based approach reaches 401 QPS. The proposed two-way filter with an extended bloom filter improves throughput to 481 QPS with a 1MB filter and 564 QPS with a 1.5MB filter. Evaluations at efs values of 40, 60, 100, and 200 demonstrate that the compact hash-based filter enhances throughput with minimal memory overhead.

Fig. 14(b) shows results for the Paper dataset with many queries. The hash-based two-way filter combined with the

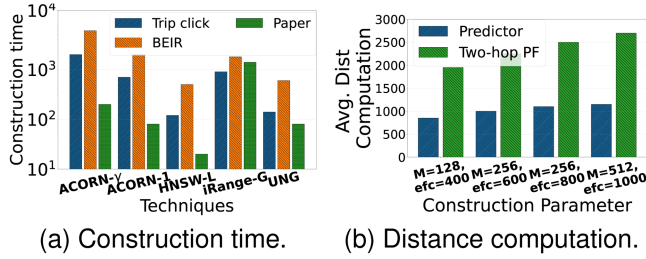


Fig. 15. Construction time and distance computation.

predictive approach achieves higher throughput than other methods. Throughput increases with filter size, as smaller filters have higher false positive rates. The in-memory filter keeps an approximate sketch of predicate presence, allowing efficient identification of attribute items in files, which reduces I/O operations and boosts throughput.

Construction time and distance computation: Fig. 15(a) presents our analysis of construction time for various hybrid search strategies, comparing original implementations against low, medium, and high embedding vector sizes. We exclude preprocessing costs for methods like iRange Graph and UNG, as well as construction costs for in-memory predictors, focusing primarily on the naïve implementation with HNSWLib.

The construction of the graphs can be categorized in two ways: 1) based on embedding vectors (ACORN- γ , ACORN-1, and HNSWLib (naïve)) and 2) based on attributes (IRange filter and UNG). The results indicate that the naïve implementation exceeds the performance of ACORN- γ and ACORN-1 by factors of 18x and 5x, respectively, for the TripClick dataset. In the case of the BEIR dataset, the performance advantage is 11x and 3x, while for the paper dataset, the naïve method is 10x and 4x faster.

In the attribute-based approaches, the naïve method demonstrates significant advantages in construction speed. It achieves speed improvements of 8x and 1.2x for TripClick, 4x and 1.4x for BEIR, and an impressive 20x and 2x advantage over the IRange graph and UNG for the paper dataset. Other methods incur considerable construction overhead, such as creating edges and additional edge pruning processes. In contrast, the naïve approach utilizes HNSW, which does not require extensive resources for construction.

Fig. 15(b) illustrates the results of the average distance computation for the predictive approach, compared to the nearest competing method, which is an exhaustive two-hop PF search with varying construction parameters with the same efc value. The results indicate that the proposed approach achieves improvements of 2x for $M=128$, 2.3x for $M=256$, and 2.4x for $M=512$ in comparison to the alternative method.

Memory consumption: Fig. 16(a) illustrates the memory consumption results for graph-based hybrid search techniques, comparing their original implementations with fixed sizes of efc and M across three datasets: high (BEIR), medium (YouTube), and low (Paper). In this experiment, we focused solely on ACORN, specifically ACORN- γ . ACORN-1 was not considered a competitor since both HNSWLib and ACORN-1 use the same HNSW graph. The results indicate that the naïve HNSW graph

consumes less memory than its competitors by factors of 1.8x, 2.6x, and 1.3x for the TripClick dataset, and by 2x, 1.4x, and 1.2x for the BEIR dataset. In the case of the Paper dataset, the factors are 1.5x, 1.2x, and 1.18x against the ACORN, IRange graph, and UNG schemes, respectively.

Fig. 16(b) illustrates the memory usage of the prediction algorithm as the cluster size varies. In this experiment, we focus solely on the TripClick dataset because the size of the prediction algorithm for a cluster remains consistent across all datasets. The graph demonstrates that the CMS sketches maintain a fixed size and require an additional 48 MB for smaller clusters. In contrast, for larger clusters, they only need an extra 8 MB to accommodate the predictive data structure. Additionally, the regression model is lightweight and only needs an extra 13 KB for smaller clusters.

For our proposed predictive algorithm, we utilize a different data structure to hold the cluster identifiers for each data vector. In this experiment, we select two datasets based on their size. It is important to note that the hash-based approach consumes 40 MB for the TripClick dataset and 90 MB for the paper dataset. In contrast, although the construction time increases for the memory block-based approach, it requires only 2 MB and 6 MB, respectively. Other schemes, such as ACORN- γ , have size requirements that increase with the size of γ . Additionally, the IRange graph and UNG methods are dependent on attribute values; as the number of distinct attributes increases, the complexity of the graph structure also increases, as shown in Fig. 16(c).

Cluster selection: In the proposed predictive approach, a newly identified node can belong to multiple clusters. Fig. 17 illustrates the impact of different cluster selection methods on QPS and recall, considering varying cluster sizes with the TripClick dataset. We examine three methods for selecting clusters: (1) a prespecified memory block for the attribute value that guides cluster selection based on maximum frequency, (2) scanning all hash sets to select the cluster with the maximum value, and (3) random cluster selection.

Fig. 17(a) shows the effects of these different cluster selection methods on QPS. We have omitted the results for random selection, as its performance is similar to that of the memory block method. The findings indicate that as the size of the cluster increases, the QPS for hash set scanning also increases, because it requires fewer clusters for scanning. However, there is a slight performance degradation for the memory block approach as the cluster size increases. Still, it remains 1.8x superior to the hash set scan for a cluster size of 100 K. Similarly, we skip the memory block for recall because recall using complete scanning is analogous to recall using a memory block. Fig. 17(b) illustrates the recall results, and it depicts that the recall rate for random cluster selection is slightly lower than that for exhaustive cluster selection.

D. Impact of Popularity Threshold

Fig. 18 shows the impact of the popularity threshold (Δ) for different selectivities across datasets, for a small $efc = 20$ and construction parameters adopted from ACORN [14]. Each point

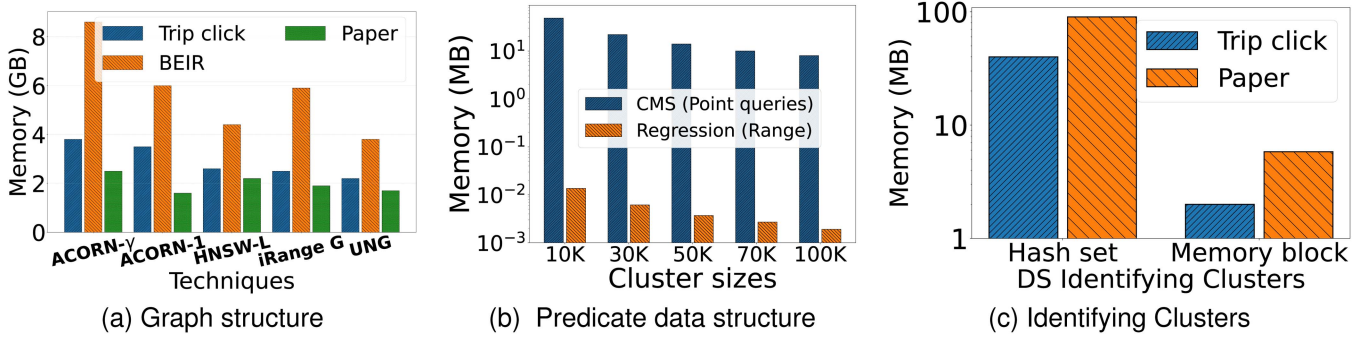


Fig. 16. Memory consumption by different data structures.

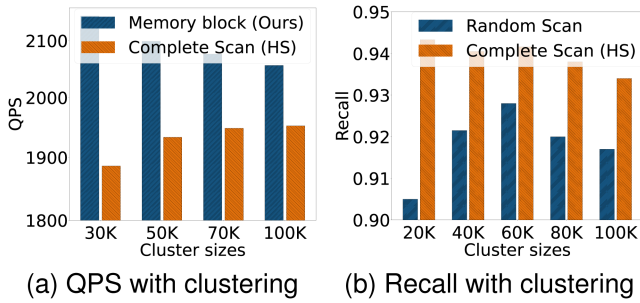


Fig. 17. Cluster selection.

in the graph corresponds to a different Δ value (e.g., 0.05, 0.1, 0.15, 0.2, 0.25, 0.3, 0.35, etc.).

Initially, we discuss the results for smaller embedding size datasets. Fig. 18(b) presents the results for the Paper dataset. The results show that for high-selectivity queries (e.g., when only 10% of the dataset records satisfy the query predicate), the QPS is higher because of fewer distance computations. However, the best trade-off between QPS and recall occurs when Δ is between 0.05 and 0.15, where both high QPS and better recall are achieved. As selectivity moves such as from 10% to 80%, recall improves, but this comes at the cost of reduced QPS. For example, at 80% query selectivity, the Δ values between 0.75 and 0.80 provide the best QPS–recall trade-off. Beyond this elbow point, recall begins to drop and gradually aligns with the performance of post-filtering. Similarly, Fig. 18(c) shows the impact of Δ on the SIFT dataset. The results reveal similar trends. For high selectivity queries, recall is lower, but QPS remains high due to fewer distance computations. The elbow point remains relatively consistent but shifts depending on the selectivity. For instance, at 10% selectivity the optimal Δ lies between 0.05 and 0.15, whereas for 80% selectivity it lies between 0.75 and 0.82.

Fig. 18(a) and (d) depict the results for the TripClick and YouTube for larger embedding size datasets. Similar trends are observed for both datasets. For high-selectivity queries, recall slightly decreases, but the system achieves higher QPS due to fewer distance computations. In contrast, low-selectivity queries require more distance computations, resulting in slightly higher recall but lower QPS. The optimal Δ values typically lie around the elbow point of the graph. For example, at 10% selectivity, the

TABLE V
PERFORMANCE COMPARISON OF REGRESSION MODELS

Model	Metric	10K	50K	100K	200K
LR	MAE	0.1147	0.0937	0.0835	0.0753
	Time (s)	9.70	6.98	5.48	4.58
PLR (4)	MAE	0.0096	0.01121	0.01312	0.01525
	Time (s)	16.29	9.88	9.23	7.18
PLR (8)	MAE	0.0021	0.0029	0.0039	0.00465
	Time (s)	18.09	11.41	9.44	7.98
PLR (12)	MAE	0.0010	0.0011	0.00112	0.00114
	Time (s)	19.21	12.60	11.64	7.99
QR	MAE	0.1062	0.0848	0.0745	0.0615
	Time (s)	15.69	8.36	6.39	6.1613

best trade-off occurs when Δ is between 0.05 and 0.15, whereas for higher selectivity queries, the optimal Δ lies between 0.75 and 0.85. These observations suggest that if the query selectivity for a predicate is known in advance, Δ should be chosen near the elbow point corresponding to that selectivity level to achieve a balanced trade-off between QPS and recall.

E. Linear Regression and Piecewise Linear Regression Model

We compare the accuracy and training cost of CDF prediction models, including simple linear regression (LR) [25], piecewise linear regression (PLR) [26] with different segment sizes (i.e., 4, 8, and 12), as well as quantile regression (QR) with median (0.5) prediction. Table V reports the mean absolute error and training/inference time for different cluster sizes on the YouTube dataset. In our case, QR achieves comparable accuracy to LR; however, it incurs a higher computational cost. It shows that simple LR is faster; however, it has lower accuracy, particularly for small cluster sizes.

In contrast, piecewise linear regression achieves higher accuracy, although it incurs additional overhead in both training and prediction time as the segment size increases. Based on these observations, we suggest using piecewise linear regression with smaller segment sizes to achieve more efficient and accurate CDF prediction.

VI. RELATED WORK

In this section, we provide a detailed discussion on related work for vector databases and filtered search.

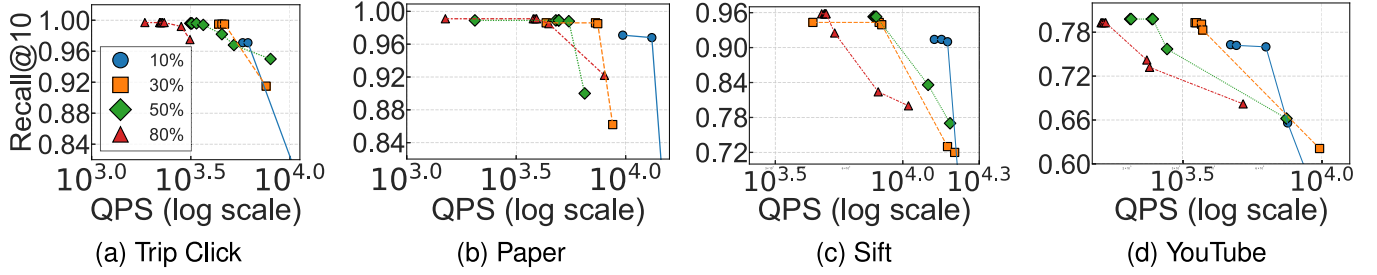


Fig. 18. Varying popularity threshold (the y-axis label in (a) is shared across all subfigures; legends are shown in (a)).

Vector databases: A vector database is designed to store high-dimensional vector embeddings. Notable open-source options include Weaviate [3], which supports hybrid semantic search, and Pinecone [35], commonly used in recommendation engines. Milvus [2] is often utilized for NLP tasks, while Meta’s FAISS [22] provides algorithms for similarity search optimized for both GPU and CPU. Qdrant [36] is also recognized for AI-driven search. All these databases enable pre-filtering before similarity searches, with Weaviate and FAISS additionally offering post-filtering options.

Filtering approaches: A recent survey by Pan et al. [5] discusses algorithms for vector similarity search and highlights filtering specific keys. AnalyticDB-V [4] creates a query processing plan based on selectivity, starting with a B-tree scan followed by an ANN search for high-selectivity queries, and prioritizes ANN for low-selectivity queries, with a subsequent filtering step. However, its PQ is lossy, potentially reducing accuracy. The stitched vama [9] constructs multiple graphs for various filters, merging them into a single graph, but increases memory usage as its size correlates with the number of filters. NPG [12] introduces the fusion distance but may face accuracy issues due to reliance on singular attributes. Cai et al. [17] explore further variations in ANN searches. ACORN [14] provides two predicate-agnostic variants: ACORN-I, which enhances HNSW for two-hop searches but may reduce efficiency, and ACORN- γ , which improves search efficiency with a query-dependent RNG-based pruning strategy. Conversely, NaviX [13] computes local selectivity online for two-hop searches, moving away from the ACORN approach. While both methods are predicate- and metadata-agnostic, they require filtering during graph traversal, potentially leading to suboptimal nearest neighbors.

Range-filtering for approximate nearest neighbor: Some studies focus on range filtering, which typically requires a separate graph for each query range, leading to a space requirement of $O(n^2 \cdot M)$ [8]. To address this, SeRF extends the HNSW graph by inserting items based on monotonically increasing attribute order [10]. Other approaches, like iRange graph [8] and WinFilter [16], utilize segment trees to create interconnected nodes that form a graph, further pruned with an RNG technique for efficiency. However, these methods are effective mainly for single-attribute datasets sorted by that attribute and still need separate indices for each attribute. We employ lightweight sketches that hold the popularity of attributes. These sketches are constructed during the formation of the HNSW graph. We chose to explore neighboring nodes only when the popularity of a particular attribute is high.

VII. CONCLUSION

In this study, we propose a solution for hybrid query processing that takes into account the popularity of metadata in making search decisions. We maintain the popularity of attributes using in-memory sketches, including the count-min sketch for point queries and a CDF-based regression model for range filtering. To begin, we construct clusters by utilizing the base layer of HNSW graphs and update the in-memory sketches with attribute values from the nodes. During the search process, we identify the nearest neighbor of the query vector and then determine whether to conduct a two-hop search based on the popularity of a specific predicate within the corresponding cluster. We performed experiments on five real-world datasets. The results show that our popularity-aware solution, along with certain optimizations, outperforms alternative approaches and exhibits greater robustness.

ACKNOWLEDGMENT

This work was initiated while the Adeel Aslam was at the University of Modena and Reggio Emilia, Italy. We acknowledge the use of AI tools in preparing this manuscript. ChatGPT and Grammarly Premium were used to improve grammar, clarity, vocabulary, and writing style.

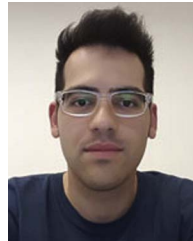
REFERENCES

- [1] J. J. Pan, J. Wang, and G. Li, “Vector database management techniques and systems,” in *Proc. Companion Int. Conf. Manage. Data*, Santiago, Chile, 2024, pp. 597–604.
- [2] M. Contributors, “Milvus documentation,” 2025. [Online]. Available: <https://github.com/milvus-io/milvus-docs>
- [3] Weaviate-VDB, “Filtered vector search | Weaviate - vector database,” 2025. [Online]. Available: <https://weaviate.io>
- [4] C. Wei et al., “AnalyticDB-V: A hybrid analytical engine towards query fusion for structured and unstructured data,” in *Proc. VLDB Endowment*, vol. 13, no. 12, pp. 3152–3165, 2020.
- [5] J. J. Pan, J. Wang, and G. Li, “Survey of vectordatabase management systems,” *VLDB J.*, vol. 33, no. 5, pp. 1591–1615, 2024.
- [6] R. Gangi Reddy, O. Attia, Y. Li, H. Ji, and S. Potdar, “AGRA ME: Any-granularity ranking with multi-vector embeddings,” in *Proc. Conf. Empirical Methods Natural Lang. Process.*, Miami, Florida, USA, 2024, pp. 8630–8641.
- [7] Y. A. Malkov and D. A. Yashunin, “Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs,” *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 42, no. 4, pp. 824–836, Apr. 2020, doi: [10.1109/TPAMI.2018.2889473](https://doi.org/10.1109/TPAMI.2018.2889473).
- [8] Y. Xu, J. Gao, Y. Gou, C. Long, and C. S. Jensen, “iRange-graph: Improving range-dedicated graphs for range-filtering nearest neighbor search,” in *Proc. ACM Manag. Data*, vol. 2, no. 6, 2024, pp. 1–26.

- [9] S. Gollapudi et al., "Filtered-DiskANN: Graph algorithms for approximate nearest neighbor search with filters," in *Proc. ACM Web Conf.*, Austin, Texas, USA, 2023, pp. 3406–3416.
- [10] C. Zuo, M. Qiao, W. Zhou, F. Li, and D. Deng, "SeRF: Segment graph for range-filtering approximate nearest neighbor search," in *Proc. ACM Manage. Data*, vol. 2, no. 1, pp. 1–26, 2024.
- [11] A. Liang, P. Zhang, B. Yao, Z. Chen, Y. Song, and G. Cheng, "Unify: Unified index for range filtered approximate nearest neighbors search," in *Proc. VLDB Endowment*, London, U.K., pp. 1118–1130, 2025.
- [12] M. Wang, L. Lv, X. Xu, Y. Wang, Q. Yue, and J. Ni, "Navigable proximity graph-driven native hybrid queries with structured and unstructured constraints," 2022, *arXiv:2203.13601*.
- [13] G. Sehgal and S. Salihoğlu, "NaviX: A native vector index design for graph DBMSs with robust predicate-agnostic search performance," in *Proc. VLDB Endowment*, London, U.K., pp. 4438–4450, 2025.
- [14] L. Patel, P. Kraft, C. Guestrin, and M. Zaharia, "ACORN: Performant and predicate-agnostic search over vector embeddings and structured data," in *Proc. ACM Manag. Data*, vol. 2, no. 3, May 2024, doi: [10.1145/3654923](https://doi.org/10.1145/3654923).
- [15] Y. Wang, Z. He, Y. Tong, Z. Zhou, and Y. Zhong, "Timestamp approximate nearest neighbor search over high-dimensional vector data," in *Proc. Int. Conf. Data Eng.*, Hong Kong, SAR, China, 2025, pp. 3043–3055.
- [16] J. Engels, B. Landrum, S. Yu, L. Dhulipala, and J. Shun, "Approximate nearest neighbor search with window filters," in *Proc. Int. Conf. Mach. Learn.*, 2024, pp. 12469–12490.
- [17] Y. Cai, J. Shi, Y. Chen, and W. Zheng, "Navigating labels and vectors: A unified approach to filtered approximate nearest neighbor search," in *Proc. ACM Manag. Data*, vol. 2, no. 6, pp. 1–27, 2024.
- [18] J. Luo, M. Qiao, C. Zuo, and D. Deng, "Tag-filtered approximate nearest neighbor search," in *Proc. Int. Conf. Data Eng.*, 2025, pp. 3642–3654.
- [19] H. Jegou, M. Douze, and C. Schmid, "Product quantization for nearest neighbor search," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 33, no. 1, pp. 117–128, Jan. 2010.
- [20] A. Gionis, P. Indyk, and R. Motwani, "Similarity search in high dimensions via hashing," in *Proc. VLDB*, M. P. Atkinson, M. E. Orłowska, P. Valduriez, S. B. Zdonik, and M. L. Brodie, Eds. Edinburgh, Scotland, Morgan Kaufmann, 1999, pp. 518–529. [Online]. Available: <http://www.vldb.org/conf/1999/P49.pdf>
- [21] J. Zobel and A. Moffat, "Inverted files for text search engines," *ACM Comput. Surv.*, vol. 38, no. 2, pp. 6–es, 2006.
- [22] Facebook Research, "Faiss," 2025. [Online]. Available: <https://github.com/facebookresearch/faiss>
- [23] S. Sanfilippo, "Scaling hnsws," 2025. [Online]. Available: <http://antirez.com/news/156>
- [24] G. Cormode and S. Muthukrishnan, "An improved data stream summary: The count-min sketch and its applications," *J. Algorithms*, vol. 55, no. 1, pp. 58–75, 2005.
- [25] T. Kraska, A. Beutel, E. H. Chi, J. Dean, and N. Polyzotis, "The case for learned index structures," in *Proc. Int. Conf. Manage. Data*, Houston, Texas, USA, 2018, pp. 489–504.
- [26] P. Ferragina et al., "The PGM-index: A fully-dynamic compressed learned index with provable worst-case bounds," in *Proc. VLDB Endowment*, vol. 13, no. 8, pp. 1162–1175, 2020.
- [27] K. Vaidya, S. Chatterjee, E. Knorr, M. Mitzenmacher, S. Idreos, and T. Kraska, "SNARF: A learning-enhanced range filter," in *Proc. VLDB Endowment*, vol. 15, no. 8, pp. 1632–1644, 2022.
- [28] G. Chen, Z. He, M. Li, and S. Luo, "Oasis: An optimal disjoint segmented learned range filter," in *Proc. VLDB Endowment*, vol. 17, no. 8, pp. 1911–1924, 2024.
- [29] W. R. Punter, O. Papapetrou, and M. Garofalakis, "OmniSketch: Efficient multi-dimensional high-velocity stream analytics with arbitrary predicates," in *Proc. VLDB Endowment*, vol. 17, no. 3, pp. 319–331, Nov. 2023.
- [30] H. Jégou, M. Douze, C. Schmid, and P. Pérez, "Aggregating local descriptors into a compact image representation," in *Proc. IEEE Comput. Soc. Conf. Comput. Vis. Pattern Recognit.*, 2010, pp. 3304–3311.
- [31] M. Wang, L. Lv, X. Xu, Y. Wang, Q. Yue, and J. Ni, "Navigable proximity graph-driven native hybrid queries with structured and unstructured constraints," 2022, *arXiv:2203.13601*.
- [32] N. Thakur, N. Reimers, A. Rücklé, A. Srivastava, and I. Gurevych, "BEIR: A heterogeneous benchmark for zero-shot evaluation of information retrieval models," in *Proc. 35th Conf. Neural Inf. Process. Syst. Datasets Benchmarks Track (Round 2)*, 2021, pp. 1–16. [Online]. Available: <https://openreview.net/forum?id=wCu6T5xFjeJ>
- [33] N. Rekabsaz, O. Lesota, M. Schedl, J. Brassey, and C. Eickhoff, "TripClick: The log files of a large health web search engine," in *Proc. 44th Int. ACM SIGIR Conf. Res. Develop. Inf. Retrieval*, 2021, pp. 2507–2513.
- [34] S. Abu-El-Haija et al., "YouTube-8M: A large-scale video classification benchmark," 2016, *arXiv:1609.08675*.
- [35] Pinecone-VDB, "Pinecone - vectordatabase," 2025. [Online]. Available: <https://www.pinecone.io/>
- [36] Qdrant-VDB, "Qdrant - vectordatabase," 2025. [Online]. Available: <https://qdrant.tech/>



Adeel Aslam received the PhD degree from the University of Modena and Reggio Emilia, Italy, in 2025. He is currently a research fellow with the University of Southampton, U.K. His research focuses on vector databases and distributed query processing. He serves on VLDB-26, VLDB-27, EDBT-27, and ICDE-26 program committees.



Luca Gagliardelli is currently an associate professor with eCampus University, Italy, and former research associate with the University of Modena and Reggio Emilia. His research interests include entity resolution and Big Data integration. He serves on VLDB and ICDE program committees.



El Kindi Rezig (Member, IEEE) is currently an assistant professor with the University of Utah and former postdoctoral associate with MIT CSAIL. His research interests include data discovery, preparation, and debugging. He serves on VLDB, SIGMOD, and ICDE program committees.



George Konstantinidis is currently a professor with the University of Southampton, U.K. and former senior researcher with the University of Oxford. He also leads several Horizon Europe projects and international research collaborations. His research interests include data management, privacy, integration, and knowledge graphs.



Giovanni Simonini is currently an associate professor with the University of Modena and Reggio Emilia. He was a postdoctoral associate with MIT CSAIL. He has authored or coauthored in top-tier venues and serves on the program committees of ACM SIGMOD, VLDB, and IEEE ICDE. His research focuses on scalable data management for AI. He was the recipient of the IEEE Computer Society Italy Chapter Best PhD Thesis Award.